# Instrucciones de despliegue (AWS)

Esta guía documenta cómo se desplegó el proyecto y cómo repetirlo. Se hizo con una cuenta nueva de **AWS (plan gratuito / Free plan)**.

## Desviaciones respecto al enunciado

| Enunciado | Lo que se usó | Motivo |
|---|---|---|
| Región N. Virginia (us-east-1) | **Ohio (us-east-2)** | El plan gratuito de la cuenta nueva solo permite operar en us-east-2 sin pasar a plan de pago. |
| Instancia t2.small | **t3.micro** (1 GB RAM) + 2 GB de swap | Es el único tipo elegible en el plan gratuito. El swap compensa la poca memoria. |
| Route 53 para el DNS | **DuckDNS** (gratuito) + HTTPS con Let's Encrypt | Evita el costo de un dominio de pago; Google exige dominio con HTTPS para el login. |

El resto del diseño (EC2, S3, base relacional, PM2, nginx, IAM, SSH) es el mismo.

## Resumen de lo desplegado

| Elemento | Valor |
|---|---|
| Región | us-east-2 (Ohio) |
| Instancia | t3.micro, Ubuntu Server 24.04 LTS, ID `i-080caf245240b78e3` |
| IP pública (Elastic IP) | `77.113.123.181` |
| Dominio | `miredsocialweb.duckdns.org` (DuckDNS, gratuito) |
| DNS público de AWS | `ec2-77-113-123-181.us-east-2.compute.amazonaws.com` (confirmar en la consola) |
| Sitio | `https://miredsocialweb.duckdns.org` |
| Frontend | React (Vite) compilado en `frontend/dist`, servido por nginx |
| Backend | Node.js/Express en el puerto 4000 (PM2), accesible solo vía nginx |
| Base de datos | PostgreSQL local en la instancia (`mi_red_social`) |
| Imágenes | Bucket S3 `mi-red-social-imagenes-meli2901` (carpeta `post/`) |
| Acceso de la app a S3 | **Rol IAM asociado a la instancia** (sin claves en el `.env`) |

## 0. Requisitos previos

- Cuenta de AWS con crédito disponible.
- Cliente SSH (Windows: PowerShell con OpenSSH).

- El código en un repositorio de GitHub **privado** del equipo.

## 1. Par de claves SSH

1. **EC2 → Pares de claves → Crear par de claves**.
2. Tipo RSA, formato `.pem`. Se descarga una sola vez: guardarlo fuera del repositorio (el `.gitignore` excluye `*.pem`).
3. Este archivo es el par de claves SSH que se entrega al catedrático por un canal privado, **nunca** en GitHub.

## 2. Lanzar la instancia EC2

1. Cambiar la región a **Ohio (us-east-2)**.
2. **EC2 → Lanzar instancia**, AMI **Ubuntu Server 24.04 LTS** (Canonical, elegible para el plan gratuito).
3. Tipo **t3.micro**, par de claves del paso 1, almacenamiento 20-30 GB gp3.
4. Security Group con reglas de entrada:

   | Tipo | Puerto | Origen |
   |---|---|---|
   | SSH | 22 | Mi IP (al entregar, restringir; ver paso 8) |
   | HTTP | 80 | 0.0.0.0/0 |
   | HTTPS | 443 | 0.0.0.0/0 (cuando exista dominio) |

   Los puertos 4000 (Node) y 5432 (PostgreSQL) **no** se abren.
5. Asignar una **Elastic IP** a la instancia para que la IP no cambie al detenerla.

## 3. Conectarse y subir el código

```
ssh -i mi-red-social-key.pem ubuntu@77.113.123.181
```

(También se puede usar **EC2 Instance Connect** desde la consola, sin SSH local.)

```
git clone https://github.com/<usuario>/<repo>.git mi_red_social
cd mi_red_social
```

Para repositorio privado se usa un token de acceso personal; revocarlo al terminar.

## 4. Ejecutar el script de bootstrap

```
chmod +x deploy/ec2-bootstrap.sh
./deploy/ec2-bootstrap.sh      # 1.ª vez: instala todo y crea .env; se detiene
nano .env                       # completar valores
./deploy/ec2-bootstrap.sh      # 2.ª vez: BD, tablas, build de React, PM2 y nginx
```

El script: crea 2 GB de swap, instala Node 20, PostgreSQL, nginx, PM2 y las librerías de `sharp`/`canvas`; compila el frontend (`VITE_API_URL=/api`); levanta la API con PM2 (arranque automático con systemd) y configura nginx para servir `frontend/dist` y reenviar `/api` e `/imagenes` al puerto 4000.

Valores mínimos del `.env` en producción:

```
PORT=4000
NODE_ENV=production
BASE_URL=http://<IP-o-dominio>
CORS_ORIGIN=http://<IP-o-dominio>
DB_HOST=localhost
DB_NAME=mi_red_social
DB_USER=miredsocial
DB_PASSWORD=<clave fuerte>
JWT_SECRET=<aleatorio de 48 bytes>
GOOGLE_CLIENT_ID=<Google Cloud>
RECAPTCHA_SITE_KEY=<reCAPTCHA Admin>
RECAPTCHA_SECRET_KEY=<reCAPTCHA Admin>
STORAGE_DRIVER=s3
AWS_REGION=us-east-2
AWS_S3_BUCKET=<nombre-del-bucket>
REKOGNITION_ENABLED=false
```

## 5. Bucket S3 y rol IAM de la instancia

1. **S3 → Crear bucket** en us-east-2, nombre único. Desmarcar "Bloquear todo el acceso público" (el feed debe mostrar las fotos).
2. **Permisos → Política de bucket**: contenido de `deploy/s3-bucket-policy.json` con el nombre del bucket (lectura pública de objetos).
3. **IAM → Políticas → Crear política (JSON)** con `deploy/iam-app-s3-policy.json` (permisos de escritura/lectura solo sobre ese bucket).
4. **IAM → Roles → Crear rol** (entidad: servicio EC2), adjuntar esa política.
5. **EC2 → Instancia → Acciones → Seguridad → Modificar rol de IAM** y asociar el rol.
6. `pm2 restart mi-red-social`. La app usa las credenciales temporales del rol: **no se guardan claves de acceso en el servidor**.
7. Subir una foto y comprobar en S3 (`post/`) los 4 archivos: original, miniatura, blanco y negro, sepia.

## 6. reCAPTCHA, dominio, HTTPS y Google Login

- **reCAPTCHA v2**: en el panel de administración se desactivó "Verificar el origen de las solicitudes de reCAPTCHA" para que funcione también al acceder por IP. Con dominio propio se puede volver a activar agregando el dominio.
- **Dominio gratuito (DuckDNS)**: se creó `miredsocialweb.duckdns.org` apuntando a la Elastic IP `77.113.123.181`. Es una desviación respecto al requisito de Route 53 (ver tabla del inicio); el servicio equivalente de Route 53 sería una zona alojada con un registro A a la misma IP.
- **HTTPS con Let's Encrypt**: el bloque de nginx usa el nombre del dominio y Certbot emite y renueva el certificado automáticamente:

  ```
          sudo sed -i 's/server_name _;/server_name miredsocialweb.duckdns.org;/'
  /etc/nginx/sites-available/mi-red-social
          sudo nginx -t && sudo systemctl reload nginx
          sudo apt install certbot python3-certbot-nginx -y
          sudo certbot --nginx -d miredsocialweb.duckdns.org --agree-tos -m <correo> --
  redirect --no-eff-email
  ```

  Requiere el puerto 443 abierto en el Security Group.
- **Actualizar la dirección de la app** en `.env` (`BASE_URL` y `CORS_ORIGIN` con `https://miredsocialweb.duckdns.org`) y `pm2 restart mi-red-social`.
- **Google Login**: usa el Client ID del equipo (`GOOGLE_CLIENT_ID` en `.env` y en `frontend/src/config.js`, que se incrusta al compilar el frontend). Google exige que el origen `https://miredsocialweb.duckdns.org` esté en "Orígenes autorizados de JavaScript" y, mientras la pantalla de consentimiento esté en modo Prueba, que cada correo esté como usuario de prueba. No acepta IPs.

## 7. Acceso del catedrático (IAM limitado)

Se creó el usuario IAM `catedratico` con la política `deploy/iam-ec2-start-stop-policy.json`: puede **ver** instancias y **iniciar/detener únicamente** la instancia `i-080caf245240b78e3` en us-east-2. Cualquier otra acción devuelve "Access Denied".

Este tipo de cuenta se creó con **clave de acceso** (CLI), porque la consola del plan gratuito no habilita contraseña de consola para usuarios IAM. Comandos de uso:

```
        aws configure           # Access key ID, Secret key, región us-east-2, formato
json
        aws ec2 describe-instances --region us-east-2
        aws ec2 start-instances --instance-ids i-080caf245240b78e3 --region us-east-2
        aws ec2 stop-instances  --instance-ids i-080caf245240b78e3 --region us-east-2
```

Detalles de la entrega en `ENTREGA_CATEDRATICO.md`.

## 8. Antes de la entrega

1. Respaldar la BD (`deploy/backup-db.sh`).

2. Cambiar la regla SSH del Security Group a una IP concreta o dejarla según se acuerde con el catedrático.
3. **Detener (Stop, no Terminate)** la instancia: `aws ec2 stop-instances --instance-ids i-080caf245240b78e3 --region us-east-2` o **EC2 → Estado de la instancia → Detener**. Con la Elastic IP asociada, la IP se conserva al reiniciar. Disco y base de datos se mantienen.
4. Verificar que el estado sea **Stopped**.

## 9. Checklist de verificación

- [ ] `http://<dominio-o-IP>/` carga la app React
- [ ] Registro e inicio de sesión con correo y reCAPTCHA
- [ ] Login con Google (requiere el dominio HTTPS autorizado)
- [ ] Subir foto: aparecen 4 archivos en S3 (`post/`)
- [ ] Análisis facial funciona (`pm2 logs mi-red-social`)
- [ ] `pm2 status` en línea y sobrevive a un reinicio de la instancia
- [ ] Usuario `catedratico` puede iniciar/detener y nada más
- [ ] Instancia en estado **Stopped** al entregar

## 10. Solución de problemas

| Síntoma | Causa probable | Solución |
|---|---|---|
| `pm2 logs` con error de BD | `.env` incorrecto o PostgreSQL detenido | `sudo systemctl status postgresql`, revisar `DB_*` |
| La instancia se congela al procesar fotos | Poca RAM (t3.micro) | Confirmar swap con `free -h`; el análisis facial tarda varios segundos |
| Subida de imagen falla con 500 | Faltan librerías de `canvas` | `sudo apt install libcairo2-dev libpango1.0-dev libjpeg-dev libgif-dev librsvg2-dev && npm rebuild` |
| nginx 502 | Node no corre | `pm2 status`, `pm2 restart mi-red-social` |
| nginx 403 al abrir el sitio | nginx sin permiso en la carpeta home | `sudo chmod o+x $HOME` |
| Fotos no se ven | Política del bucket o rol IAM | Revisar política pública del bucket y el rol asociado a la instancia |
| reCAPTCHA "dominio no válido" | Verificación de origen activa con IP | Desactivarla o usar el dominio registrado |
| Error 400 al usar Google | Origen no permitido (IP) | Usar dominio con HTTPS autorizado en Google Cloud |